SYSTEM CALLS (Exec.c)

INPUT:

user07@jannani:~/systemcalls — /usr/bin/vim exec.c

```
#include<stdio.h>
#include<unistd.h>
int main()
{
        printf("before exec()\n");
        execl("/bin/ls","ls",NULL);
        printf("After exec()\n");
        return 0;
}
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"exec.c" 9L, 147B                                    9,1           All
```

OUTPUT:

user07@jannani:~/systemcalls

```
[user07@jannani ~]$ ls
 Ab        c                 Downloads   pc          sample.txt   Videos
 abc.sh    Cpu               fcfs.c      pet         Scheduling   while.sh
 ab.sh    'CPU Scheduling'   if          Pictures    self.sh
 a.out     Desktop           lora        Public      Templates
 bankers   Documents         Music       sample.c    test
[user07@jannani ~]$ mkdir systemcalls
[user07@jannani ~]$ cd systemcalls
[user07@jannani systemcalls]$ vi exec.c
[user07@jannani systemcalls]$ gcc exec.c
[user07@jannani systemcalls]$ ./a.out
before exec()
a.out  exec.c
[user07@jannani systemcalls]$
```

Fork.c

Input:

user07@jannani:~/systemcalls — /usr/bin/vim fork.c

```c
#include<stdio.h>
#include<unistd.h>
int main(){
        if(fork()==0){
                printf("This is child process");
        }
        else{
                printf("This is parent process\n");
        }
        printf("PID: %d\n",getpid());
        return 0;
}
~
~
~
~
~
~
~
```

Output:

user07@jannani:~/systemcalls

```
[user07@jannani ~]$ ls
 Ab        c                Downloads   pc          sample.txt   Videos
 abc.sh    Cpu              fcfs.c      pet         Scheduling   while.sh
 ab.sh    'CPU Scheduling'  if          Pictures    self.sh
 a.out     Desktop          lora        Public      Templates
 bankers   Documents        Music       sample.c    test
[user07@jannani ~]$ mkdir systemcalls
[user07@jannani ~]$ cd systemcalls
[user07@jannani systemcalls]$ vi exec.c
[user07@jannani systemcalls]$ gcc exec.c
[user07@jannani systemcalls]$ ./a.out
before exec()
a.out  exec.c
[user07@jannani systemcalls]$ vi exec.c
[user07@jannani systemcalls]$ vi fork.c
[user07@jannani systemcalls]$ gcc fork.c
[user07@jannani systemcalls]$ ./a.out
This is parent process
PID: 2934
This is child processPID: 2935
[user07@jannani systemcalls]$
```

Opendie.c

Input:

```
user07@jannani:~/systemcalls — /usr/bin/vim opendir.c
#include<stdio.h>
#include<dirent.h>
int main(){
        DIR *dir;
        dir = opendir(".");
        if(dir==NULL){
                printf("Failed to open directory\n");
                return 1;
        }
        printf("Directory opened successfully!\n");
        closedir(dir);
        return 0;
}
~
~
~
~
~
~
~
~
~
~
-- INSERT --                                    2,11          All
```

Output:

```
compilation terminated.
[user07@jannani systemcalls]$ vi opendir.c
[user07@jannani systemcalls]$ gcc opendir.c
[user07@jannani systemcalls]$ ./a.out
Directory opened successfully!
[user07@jannani systemcalls]$
```

Pid.c

Input:

user07@jannani:~/systemcalls — /usr/bin/vim pid.c

```c
#include<stdio.h>
#include<unistd.h>
int main(){
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n",getppid());
        return 0;
}
~
~
~
~
~
~
```

Output:

user07@jannani:~/systemcalls

```
[user07@jannani systemcalls]$ vi pid.c
[user07@jannani systemcalls]$ gcc pid.c
[user07@jannani systemcalls]$ ./a.out
PID: 3407
PPID: 2628
[user07@jannani systemcalls]$
```

readdir.c

Input:

user07@jannani:~/systemcalls — /usr/bin/vim readdir.c

```
#include<stdio.h>
#include<dirent.h>
int main(){
        DIR *dir;
        struct dirent *entry;
        dir=opendir(".");
        if(dir==NULL){
                printf("Cannot open directory\n");
                return 1;
        }
        printf("Files in directory:\n");
        while((entry=readdir(dir))!=NULL){
                printf("%s\n",entry->d_name);
        }
        closedir(dir);
        return 0;
}
~
~
~
~
~
~
"readdir.c" 17L, 304B                                   12,35-42      All
```

Output:

user07@jannani:~/systemcalls

```
[user07@jannani systemcalls]$ gcc readdir.c
[user07@jannani systemcalls]$ ./a.out
Files in directory:
.
..
exec.c
fork.c
opendir.c
pid.c
readdir.c
a.out
[user07@jannani systemcalls]$
```